数据结构：第七章散列表
复习讲义

目录

1 概述与动机 (Motivation) 2
1.1 从搜索问题说起 . . . 2
1.2 符号表 ADT (Symbol Table ADT) . . . 2
1.3 哈希表结构 . . . 3

2 哈希函数 (Hash Functions) 3
2.1 设计原则 . . . 3
2.2 整数键的哈希 . . . 3
2.3 字符串键的哈希 . . . 4

3 分离链接法 (Separate Chaining) 5
3.1 基本思路 . . . 5
3.2 初始化 . . . 5
3.3 查找 (Find) . . . 5
3.4 插入 (Insert) . . . 6
3.5 性能分析 . . . 6

4 开放地址法 (Open Addressing) 6
4.1 线性探测 (Linear Probing) . . . 7
4.2 平方探测 (Quadratic Probing) . . . 7
4.3 双重散列 (Double Hashing) . . . 9
4.4 三种探测方法对比 . . . 10

5 再散列 (Rehashing) 10
5.1 为什么需要再散列 . . . 10
5.2 再散列过程 . . . 10
5.3 触发时机 . . . 10
5.4 复杂度分析 . . . 10

6 Robin Hood 散列 (Robin Hood Hashing) 11

6.1 核心思想：劫富济贫 11

6.2 插入实现 11

6.3 性能特点 12

7 各方法横向对比 12

8 全章总结 13

1 概述与动机 (Motivation)

1.1 从搜索问题说起

二分查找在有序数组上需要 $O(\log N)$ 时间，而将数据排好序又需要 $O(N \log N)$ 时间——有没有办法更快?

插值查找 (Interpolation Search)

对已排序表 $f[l].key,\ f[l+1].key,\ \ldots,\ f[u].key$，利用线性插值估计目标位置：

$$i = l + \frac{\texttt{key} - f[l].\texttt{key}}{f[u].\texttt{key} - f[l].\texttt{key}} \times (u - l)$$

- 对**均匀分布**的数据，期望探测次数为 $O(\log \log N)$。
- 缺点：对非均匀分布效果差；仍依赖比较操作。

散列表（哈希表）的思路更进一步：**完全不依赖比较**，而是通过一个函数直接计算出元素的存储位置，理想情况下实现 $O(1)$ 的查找、插入和删除。

1.2 符号表 ADT (Symbol Table ADT)

符号表（也叫字典）存储若干 $\langle$`name`, `attribute`$\rangle$ 对，其中 name 唯一。

操作	说明
`SymTab Create(TableSize)`	创建大小为 TableSize 的空符号表
`Boolean IsIn(symtab, name)`	查询 name 是否存在
`Attribute Find(symtab, name)`	返回 name 对应的属性
`SymTab Insert(symtab, name, attr)`	插入键值对
`SymTab Delete(symtab, name)`	删除键值对

1.3 哈希表结构

基本定义

哈希表 (Hash Table) `ht[]` 由 b 个**桶 (bucket)** 组成，每个桶包含 s 个**槽 (slot)**。
对每个标识符 x，**哈希函数 (hash function)** $f(x)$ 给出 x 在 `ht[]` 中的桶下标：

$$f : \{x\} \to \{0, 1, \ldots, b-1\}$$

关键参数：

- T：x 所有可能取值的数量（全集大小）
- n：当前哈希表中的标识符数量
- **标识符密度 (Identifier density)**：n/T
- **装载密度 (Loading density)**：$\lambda = n/(s \cdot b)$

冲突 (Collision) 与溢出 (Overflow)：

- **冲突**：两个不同标识符 $i_1 \neq i_2$ 哈希到同一桶：$f(i_1) = f(i_2)$。
- **溢出**：向一个已满的桶中插入新标识符。
- 当 $s = 1$（每桶一槽）时，冲突与溢出同时发生。

理想性能

若无溢出：$T_{\text{search}} = T_{\text{insert}} = T_{\text{delete}} = O(1)$

2 哈希函数 (Hash Functions)

2.1 设计原则

一个好的哈希函数应满足：

1. **计算代价低**：$f(x)$ 必须容易快速计算。
2. **冲突少**：尽可能将键均匀分布到所有桶。
3. **无偏性 (Uniform)**：对任意 x 和任意 i，有 $P(f(x) = i) = 1/b$。

2.2 整数键的哈希

$$f(x) = x \ \% \ \texttt{TableSize}$$

TableSize 应选素数

若 `TableSize = 10` 而所有键都是 10 的倍数，则所有键都哈希到桶 0，完全退化。**TableSize 应选素数**，可使键在各桶间均匀分布（适合随机整数键）。

2.3 字符串键的哈希

方法一：ASCII 码之和

$$f(x) = \left(\sum_{i} x[i] \right) \ \% \ \texttt{TableSize}$$

问题：当 `TableSize = 10007`，字符串长度 ≤ 8，$x[i] \in [0, 127]$ 时，$f(x) \in [0, 1016]$，只用到了前 1016 个桶，**分布极不均匀**。

方法二：加权多项式

$$f(x) = \left(x[0] + x[1] \cdot 27 + x[2] \cdot 27^2 \right) \ \% \ \texttt{TableSize}$$

考虑前三个字符，总组合数 $= 26^3 = 17576$，但实际英文单词的前三字符组合数 < 3000，仍有大量空桶浪费。

方法三：移位散列（推荐）

$$f(x) = \left(\sum_{i=0}^{N-1} x[N-1-i] \cdot 32^i \right) \ \% \ \texttt{TableSize}$$

等价地，用 Horner 法则配合位移加速（$\times 32$ 等价于左移 5 位）：

```
Index Hash3( const char *x, int TableSize )
{
    unsigned int HashVal = 0;
    while ( *x != '\0' )
        HashVal = ( HashVal << 5 ) + *x++;
    return HashVal % TableSize;
}
```

代码关键点

- `<< 5` 比 `* 32` 快（位移比乘法快）。
- 若字符串过长（如街道地址），早期字符会被左移出有效位，可考虑只取部分字符参与计算。

3 分离链接法 (Separate Chaining)

3.1 基本思路

对哈希到同一桶的所有键，**用链表维护**，链表头存储在哈希表数组中。

```
struct ListNode {
    ElementType  Element;
    Position     Next;
};
typedef Position List;

struct HashTbl {
    int    TableSize;
    List  *TheLists;   /* Array of linked lists */
};
typedef struct HashTbl *HashTable;
```

3.2 初始化

```
HashTable InitializeTable( int TableSize )
{
    HashTable H;
    int i;
    H = malloc( sizeof( struct HashTbl ) );
    H->TableSize = NextPrime( TableSize );   /* Must be prime */
    H->TheLists = malloc( sizeof(List) * H->TableSize );
    for ( i = 0; i < H->TableSize; i++ ) {
        H->TheLists[i] = malloc( sizeof(struct ListNode) );
        H->TheLists[i]->Next = NULL;        /* Header node */
    }
    return H;
}
```

3.3 查找 (Find)

```
Position Find( ElementType Key, HashTable H )
{
    Position P;
    List L;
    L = H->TheLists[ Hash( Key, H->TableSize ) ];
    P = L->Next;
    while ( P != NULL && P->Element != Key )
        P = P->Next;
    return P;   /* NULL if not found */
```

```
}
```

与通用链表 ADT 的 Find 完全相同，只是先用哈希函数定位到对应链表。

3.4 插入 (Insert)

```
void Insert( ElementType Key, HashTable H )
{
    Position Pos, NewCell;
    List L;
    Pos = Find( Key, H );
    if ( Pos == NULL ) {       /* Key not found, insert */
        NewCell = malloc( sizeof(struct ListNode) );
        L = H->TheLists[ Hash( Key, H->TableSize ) ];
        NewCell->Next = L->Next;
        NewCell->Element = Key;
        L->Next = NewCell;     /* Insert at front of list */
    }
}
```

实用建议

使 TableSize 约等于预期键数（即让装载密度 $\lambda \approx 1$），可使每条链表平均只有 1 个节点，查找时间接近 $O(1)$。

3.5 性能分析

- 查找时间：遍历长度约 λ 的链表，期望 $O(1+\lambda)$。
- 空间：需要额外 $O(N)$ 的指针开销（链表节点）。
- 删除：直接从链表中移除节点，简单高效，无特殊处理。

4 开放地址法 (Open Addressing)

思路：不使用额外链表，冲突时在哈希表内**寻找另一个空槽**。
通用插入算法：

```
index = hash( key );
initialize i = 0;          /* collision counter */
while ( collision at index ) {
    index = ( hash(key) + f(i) ) % TableSize;
    i++;
}
```

其中 $f(0)=0$，$f(i)$ 为**冲突解决函数**。

实用建议

开放地址法中，装载密度应保持 $\lambda < 0.5$，否则性能急剧下降。

4.1 线性探测 (Linear Probing)

$$f(i) = i$$

即冲突时依次向后找：位置序列为 $h(x),\ h(x)+1,\ h(x)+2,\ \ldots$（均 mod TableSize）。

示例

将 11 个 C 库函数（acos, atoi, char, define, exp, ceil, cos, float, atol, floor, ctim）哈希到 $b=26$ 个桶中（$s=1$）：

- 装载密度 $\lambda = 11/26 \approx 0.42$
- 实测平均查找次数 $= 41/11 \approx 3.73$（远高于理论值）

主聚集问题 (Primary Clustering)

主聚集

一旦某个区域形成连续占用块（"簇"），任何哈希到簇内任意位置的新键都会将簇向后延伸，使簇越来越长。这种正反馈效应称为**主聚集**，导致实际探测次数远超理论值。

4.2 平方探测 (Quadratic Probing)

$$f(i) = i^2$$

位置序列为 $h(x),\ h(x)+1,\ h(x)+4,\ h(x)+9,\ \ldots$（均 mod TableSize）。

定理（平方探测的可用性）

若使用平方探测，且 `TableSize` 为素数，则当哈希表**至少半空**时，新元素一定能成功插入。

证明：只需证前 $\lfloor \texttt{TableSize}/2 \rfloor$ 个探测位置互不相同。
反设 $0 < i \neq j \leq \lfloor \texttt{TableSize}/2 \rfloor$ 时发生碰撞：

$$h(x) + i^2 \equiv h(x) + j^2 \pmod{\texttt{TableSize}}$$

$$\Longrightarrow\ i^2 \equiv j^2 \pmod{\texttt{TableSize}}$$

$$\Longrightarrow (i+j)(i-j) \equiv 0 \pmod{\texttt{TableSize}}$$

由于 TableSize 为素数，$(i+j)$ 或 $(i-j)$ 必须被 TableSize 整除。但：

- $0 < i+j \leq \texttt{TableSize} - 1 < \texttt{TableSize}$，故 $(i+j)$ 不能被整除；
- $|i-j| < \texttt{TableSize}/2 < \texttt{TableSize}$，且 $i \neq j$，故 $(i-j)$ 也不能被整除。

矛盾。因此前 $\lfloor \texttt{TableSize}/2 \rfloor$ 个位置确实互不相同，只要表至少半空，就能找到空槽。□

补充说明

若 TableSize 为 $4k+3$ 形式的素数，则 $f(i) = \pm i^2$ 可以探测**整张表**（不仅仅是一半）。

查找实现

```
Position Find( ElementType Key, HashTable H )
{
    Position CurrentPos;
    int CollisionNum;
    CollisionNum = 0;
    CurrentPos = Hash( Key, H->TableSize );
    while ( H->TheCells[CurrentPos].Info != Empty &&
            H->TheCells[CurrentPos].Element != Key ) {
        CurrentPos += 2 * ++CollisionNum - 1;  /* f(i)=f(i-1)+2i-1 */
        if ( CurrentPos >= H->TableSize )
            CurrentPos -= H->TableSize;         /* Avoid % */
    }
    return CurrentPos;
}
```

代码关键点

- 利用递推式 $f(i) = f(i-1) + 2i - 1$ 增量更新位置，避免每次都计算平方。
- CurrentPos -= TableSize 代替 % TableSize，位运算更快。
- 循环条件：Info == Empty（空槽）时终止；遇到 Deleted（已删除标记）时**不能**终止，必须继续探测。

插入实现

```
void Insert( ElementType Key, HashTable H )
{
    Position Pos;
    Pos = Find( Key, H );
    if ( H->TheCells[Pos].Info != Legitimate ) {
```

```
            H->TheCells[Pos].Info = Legitimate;
            H->TheCells[Pos].Element = Key;
        }
    }
```

删除与惰性删除

开放地址法中**不能直接清空**已删除的槽，否则会截断其他键的探测链，导致后续 Find 误认为键不存在。

惰性删除 (Lazy Deletion)

槽的状态 `Info` 有三种：

- `Empty`：从未使用，Find 时遇到此状态可终止探测。
- `Legitimate`：有效键，正常参与查找。
- `Deleted`：已删除的标记，Find 时**必须跳过继续探测**，Insert 时可重用此槽。

注意：频繁删除后再插入，会导致大量 Deleted 槽拖慢查找；此时应考虑**再散列**。

次聚集 (Secondary Clustering)

平方探测消除了主聚集，但引入**次聚集**：**哈希到同一初始位置的所有键，会经历完全相同的探测序列**，仍然相互干扰。双重散列可解决此问题。

4.3 双重散列 (Double Hashing)

$$f(i) = i \cdot \texttt{hash}_2(x)$$

引入第二个哈希函数，使每个键的步长不同，从而打破次聚集。

推荐的 hash_2 设计

$$\texttt{hash}_2(x) = R - (x \ \% \ R)$$

其中 R 为小于 `TableSize` 的素数。
要求：

- $\texttt{hash}_2(x) \neq 0$（步长不能为 0，否则原地死循环）；
- 保证所有槽都能被探测到。

模拟实验表明，正确实现双重散列的期望探测次数与**随机冲突解决策略**几乎相同，是开放地址法中性能最优的方案。

4.4 三种探测方法对比

方法	$f(i)$	聚集问题	性能
线性探测	i	主聚集（严重）	实际远差于理论
平方探测	i^2	次聚集（轻微）	接近理论值，实用首选
双重散列	$i \cdot h_2(x)$	无聚集	最优，接近随机

5 再散列 (Rehashing)

5.1 为什么需要再散列

平方探测要求 $\lambda < 0.5$。当哈希表超过半满时：

- 插入可能失败（找不到空槽）；
- 即使能插入，性能也急剧下降。

5.2 再散列过程

再散列步骤

1. 建立一张约**两倍大小**的新哈希表（下一个大于 $2 \times$ `TableSize` 的素数）；
2. 扫描原哈希表中所有**非删除**的有效元素；
3. 用**新的哈希函数**（基于新 TableSize）将这些元素重新插入新表。

5.3 触发时机

- 表装满一半时立即触发；
- 插入失败时触发；
- 装载密度达到某个预设阈值时触发（最灵活）。

5.4 复杂度分析

- 单次再散列：需扫描 N 个元素并重新插入，$T(N) = O(N)$。
- **摊还分析**：每次再散列之前至少已进行了 $N/2$ 次插入，因此每次插入的摊还代价为 $O(N)/(N/2) = O(1)$，整体摊还性能不受影响。

注意

在**交互式系统**中，触发再散列的那次插入会遭遇明显的延迟——对用户可见。可考虑**渐进再散列**（每次插入时顺便迁移少量元素）来平滑延迟。

6 Robin Hood 散列 (Robin Hood Hashing)

Robin Hood Hashing 由 Pedro Celis 于 1986 年提出，是对**线性探测**的改进，目标是**降低查找代价的方差**（让不同键的探测次数更均匀）。

6.1 核心思想：劫富济贫

位移量 (Displacement)

定义键 x 的**位移量**：

$$d(x) = \big(g(x) - h(x)\big) \ \%\ \texttt{TableSize}$$

其中 $g(x)$ 为 x 的**实际存储位置**，$h(x)$ 为 x 的哈希值（理想位置）。
$d(x)$ 表示 x 被迫从理想位置”移走”了多少步：

- **富键 (rich key)**：d 小，接近理想位置；
- **穷键 (poor key)**：d 大，离理想位置很远。

插入新键时，若当前探测位置已被一个”富键”占据，则**将富键驱逐出去**（将其存储位置让给新键），新键继承该位置，富键继续向后寻找空槽——即”劫富济贫”。

6.2 插入实现

```
bool RobinHood_Insert( ElementType Key, HashTable H )
{
    Position Pos = Hv = Hash( Key, H->TableSize );
    int CurrentDisp = 0;
    ElementType currentKey = Key;

    while ( true ) {
        if ( H->TheCells[Pos].Info != Legitimate ) {
            /* Empty or deleted: place here */
            H->TheCells[Pos].Info    = Legitimate;
            H->TheCells[Pos].Element = currentKey;
            H->TheCells[Pos].Disp    = CurrentDisp;
            return true;
        }
        if ( CurrentDisp > H->TheCells[Pos].Disp ) {
            /* Rob the rich: swap currentKey with stored key */
            swap( &currentKey, &H->TheCells[Pos].Element );
            swap( &CurrentDisp, &H->TheCells[Pos].Disp );
        }
        Pos = ( Pos + 1 ) % H->TableSize;
        if ( Pos == Hv ) return false;    /* Table is full */
```

```
            CurrentDisp++;
        }
    }
```

代码关键点

- 每个槽需额外存储 `Disp`（位移量），用于与当前插入键的位移量比较。
- 若 `CurrentDisp > H->TheCells[Pos].Disp`，说明当前位置的键是”富键”（离理想位近），而新键是”穷键”——抢占位置，驱逐富键。
- 被驱逐的富键继续向后探测，过程同样按 Robin Hood 规则进行。

6.3 性能特点

特性	线性探测	Robin Hood 探测
渐进复杂度	与线性探测相同	与线性探测相同
期望探测次数	较高（受聚集影响）	略低（方差更小）
最坏情况查找	$O(N)$	$O(\log N)$
高装载因子容忍	性能急剧下降	显著改善
插入速度	快	较慢（需比较位移量 + 交换）
删除复杂度	惰性删除	更复杂（需后向移位）
额外存储	无	每个槽需存储 `Disp`

适用场景

Robin Hood 散列适合**高装载因子**（λ 接近 1）且**查找频繁、插入较少**的场景，例如只读哈希表或构建后固定的哈希表。

7 各方法横向对比

方法	额外空间	λ 限制	删除	聚集
分离链接	$O(N)$ 链表	无（$\lambda \approx 1$）	简单（移除节点）	无
线性探测	$O(1)$	< 0.5	惰性删除	主聚集
平方探测	$O(1)$	< 0.5（素数表）	惰性删除	次聚集
双重散列	$O(1)$	< 0.5	惰性删除	无
Robin Hood	$O(1)$+`Disp`	可更高	复杂	无（但方差小）

选型建议

- **不限额外内存、需要简单实现**：分离链接法，删除容易，装载因子不受严格限制。
- **内存紧张、删除不频繁**：平方探测，实现简单、无主聚集，是开放地址法的实用首选。
- **追求最优查找性能、无聚集**：双重散列，代价是需要设计第二个哈希函数。
- **高装载、查找密集**：Robin Hood 散列，最坏情况 $O(\log N)$，高 λ 下更稳定。

8 全章总结

1. **哈希表的核心优势**：通过哈希函数将键直接映射到存储位置，理想情况下 $O(1)$ 完成查找、插入、删除，突破了基于比较的 $\Omega(\log N)$ 下界（代价是不支持有序遍历）。
2. **哈希函数设计**：整数键用 $x\%$`TableSize`（TableSize 取素数）；字符串键推荐移位散列（Horner 法则 + 左移 5 位），避免直接求和带来的值域不足问题。
3. **冲突解决策略**：
   - **分离链接**：链表挂在桶上，删除简单，$\lambda \approx 1$ 即可；代价是指针开销。
   - **线性探测**：简单但有主聚集，性能退化严重，不推荐。
   - **平方探测**：消除主聚集，定理保证素数表 + $\lambda < 0.5$ 时总能插入；惰性删除。
   - **双重散列**：消除次聚集，性能最优，需第二哈希函数 $h_2(x) = R - (x\%R)$。
4. **再散列**：当 λ 超限或插入失败时，构建 2× 大小的新表并重新哈希所有元素；单次 $O(N)$，摊还 $O(1)$。
5. **Robin Hood 散列**：在线性探测基础上，通过”劫富济贫”（交换位移量大/小的键）降低探测次数的方差，最坏查找 $O(\log N)$，适合高装载场景，但插入和删除更复杂。
6. **哈希表的局限**：不支持有序操作（如求最大值、排序遍历）；性能依赖哈希函数质量；最坏情况可退化到 $O(N)$（所有键冲突在同一桶）。

”哈希以 $O(1)$ 换来无序；选择冲突解决策略时，需在空间、装载因子与删除复杂度之间权衡。”